

AQUACOD: Agentic Quantum Optimization via Coordinate Descent for Scalable Drone Scheduling

Van-Quang-Huy Nguyen^{1, 5*}, Hoang-Quan Nguyen^{1, 5},
Samee U. Khan², Ilya Safro³, Hugh Churchill⁴, Khoa Luu^{1, 5}

¹Department of EECS, University of Arkansas, Fayetteville, AR, USA.

²Department of ECE, Kansas State University, Manhattan, KS, USA.

³Department of CIS, University of Delaware, Newark, DE, USA.

⁴Department of Physics, University of Arkansas, Fayetteville, AR, USA.

⁵Quantum AI Lab, University of Arkansas, Fayetteville, AR, USA.

<https://uark-cviu.github.io/projects/AQUACOD>.

*Corresponding author(s). E-mail(s): hvn001@uark.edu;
Contributing authors: hn016@uark.edu; sameekhan@ksu.edu;
isafro@udel.edu; hchurch@uark.edu; khoaluu@uark.edu;

Abstract

Quantum computing has emerged as a potential tool for solving combinatorial optimization problems, such as drone scheduling. Quantum methods typically transform the problem into a quadratic unconstrained binary optimization (QUBO) form before solving it. However, a notable limitation of this transformation is its requirement that the problem be unconstrained, which drone scheduling and many other problems do not naturally satisfy. A common workaround is to embed the constraints in the objective function via penalty terms, thereby yielding an equivalent unconstrained problem. The remaining challenge is that the appropriate penalty coefficient varies significantly from one problem to another. To determine it, we propose an agentic framework where large language model (LLM)-based agents automatically discover and design penalty functions for drone scheduling. The agents iteratively propose, evaluate, and refine them in response to solver feedback, reducing reliance on manual tuning. The experimental results show that the agentic method improves the solution, demonstrating that agentic design is a viable alternative to manual tuning.

Keywords: Drone scheduling, coordinate descent, quantum optimization, quantum computing, agentic AI

1 Introduction

Quantum computing has shown remarkable capabilities. Quantum algorithms theoretically reduce the computational complexity of many problems compared to classical ones, such as Grover's algorithm [1] for the unstructured search problem or Shor's algorithm [2] for the hidden subgroup problem. This shows that once quantum computing matures, many problems previously considered infeasible will become more tractable. Many works [3–10] have applied quantum computing to problems across a range of domains. Moreover, quantum computing shows its potential in solving many optimization problems, especially combinatorial ones [11, 12]. This is particularly significant because many logistics problems fall into this category, such as the vehicle routing problem and its variants [13]. A notable subclass is unmanned aerial vehicle (UAV) or drone problems, which play an increasingly large role in logistics, especially in last-mile delivery. It is because of the growing demand for operating very large-scale fleets of drones across multiple hubs. These problems are typically divided into two categories, i.e., routing and scheduling, as illustrated in Fig. 1. Quantum methods have been investigated for both [14–16], demonstrating the capability of quantum computing in this domain.

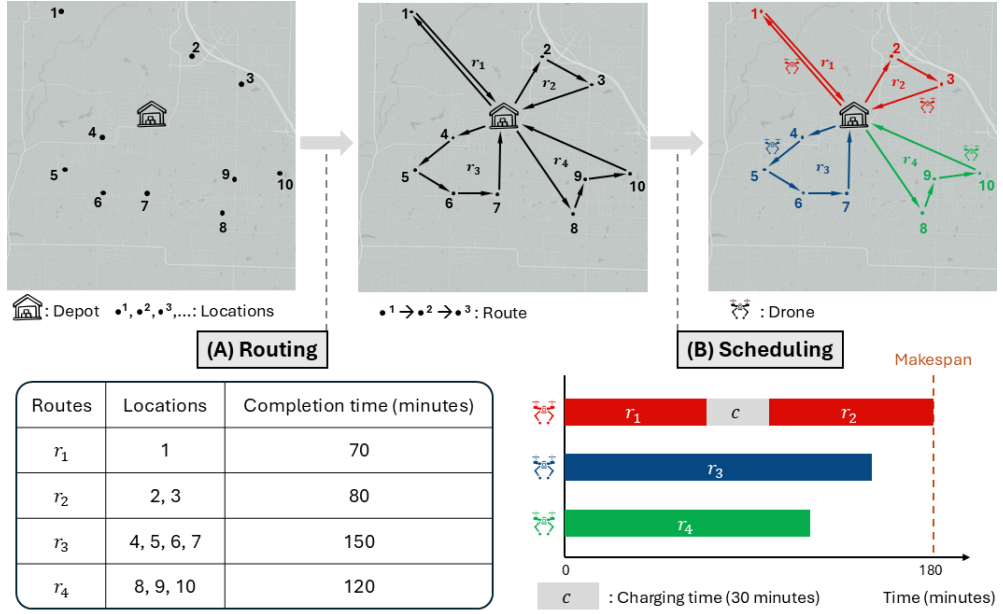


Fig. 1 The two main optimization problems in drone delivery. (A) Routing: four routes r_1 , r_2 , r_3 , and r_4 start and end at the depot, cover all locations, and satisfy the drone operation constraints. Their completion times are 70, 80, 150, and 120 minutes, respectively, minimizing the sum of the route completion times. (B) Scheduling: the red drone is assigned r_1 and r_2 (completion time: $70 + 30 + 80 = 180$ minutes), the blue drone is assigned r_3 (150 minutes), and the green drone is assigned r_4 (120 minutes), giving a minimal makespan of 180 minutes.

In parallel, artificial intelligence (AI) has been developing rapidly. AI models are being integrated ever more deeply into not only industry and daily life, but also the research process itself, accelerating the pace and reach of scientific discovery. In relation to quantum computing, AI is involved in two directions, namely quantum for AI and AI for quantum [17]. Quantum for AI uses quantum hardware to solve problems in AI, while AI for quantum uses AI to improve components in a quantum pipeline such as algorithms, circuits, or error correction codes. In this work, we focus on AI for quantum. The recent emergence of large language models (LLMs) and their diverse applications [18–20] suggests new approaches in this direction, particularly for design tasks currently performed manually by human experts.

Our contributions in this Work: We propose a novel framework that uses LLM-based agents to discover and design penalty-coefficient functions for the quadratic unconstrained binary optimization (QUBO) form of the drone scheduling problem. Choosing a penalty coefficient is typically a matter of trial and error: raise it until the constraints hold, lower it until the objective survives. This works for a single problem instance, but the tuning must be repeated whenever the parameters change. A penalty function is therefore necessary to map problem parameters to a coefficient. However, finding good ones is not trivial, since the space of candidate functions is unbounded. In our preliminary work, QUACOD [21] faced the same problem, so we chose a hand-crafted penalty function without a clear methodological explanation. Our framework overcomes this limitation by using agents to analyze and propose functions automatically, rather than tuning them manually. Experimental results show that the penalty functions proposed by agents improve up to 36 cases while degrading only 11. This provides initial evidence that agents can automate component design in quantum optimization algorithms.

This article is organized as follows. Section 2 presents the background of the proposed method. Section 3 describes how the proposed method is implemented in detail. Section 4 presents the experiments and results used to evaluate the method. Finally, Section 5 discusses what the method achieves and the limitations to be addressed in future work.

2 Background

In this section, we provide an overview of the problem and its related components. We first introduce the models and algorithms central to quantum optimization. We then present drone scheduling, the main problem, along with QUACOD [21], the state-of-the-art quantum method we previously proposed to solve it. Finally, we survey agentic methods and their potential for algorithmic discovery and design.

2.1 Quantum Optimization

2.1.1 Quadratic Unconstrained Binary Optimization

In quantum optimization, not every problem can be solved directly. Quantum methods are suitable for only specific models, most notably QUBO [11, 12]. A QUBO model is

defined as follows:

$$\underset{x_1, \dots, x_n \in \{0,1\}}{\text{minimize}} \sum_{1 \leq i < j \leq n} a_{i,j} x_i x_j + \sum_{1 \leq i \leq n} b_i x_i, \quad (1)$$

where $x_1, \dots, x_n$ are binary variables, and $a_{i,j}$ and b_i are real coefficients. The name QUBO reflects these properties: the objective is quadratic (degree at most two), unconstrained (no constraints), over binary variables.

Many combinatorial optimization problems can be reformulated as QUBO. A well-known one is the max-cut problem, which finds a partition of the vertices of a graph into two subsets such that the edges connecting the two subsets have the maximum sum of weights. This problem can be transformed directly into QUBO and has been shown to be solvable by quantum algorithms [12]. However, some problems cannot be transformed directly, such as the traveling salesman problem, and require additional techniques, most commonly penalty methods [11].

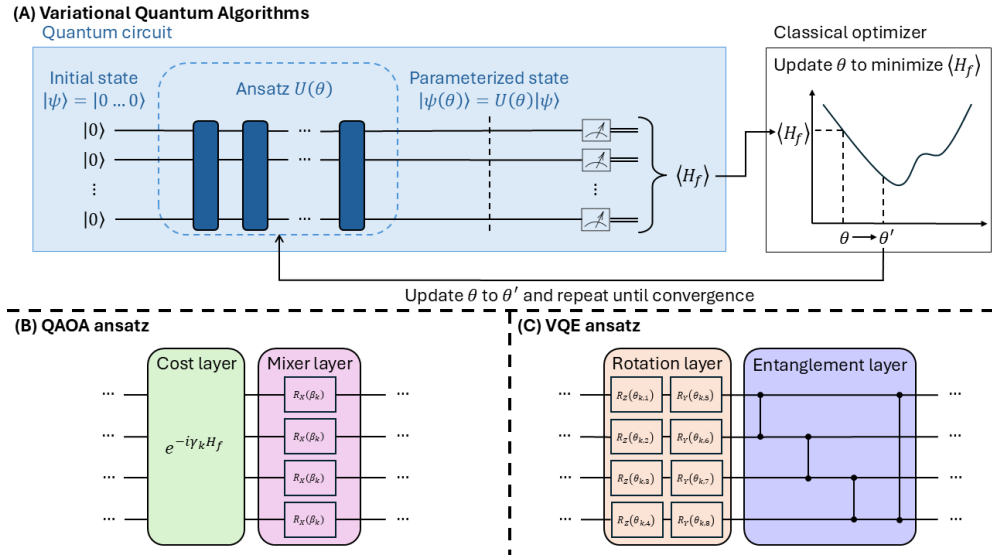


Fig. 2 Variational quantum algorithms (VQAs) and the ansatzes of QAOA [12] and VQE [22]. (A) A VQA generally consists of a quantum circuit and a classical optimizer, where the ansatz of the quantum circuit can be QAOA or VQE. (B) In the QAOA ansatz, cost and mixer layers alternate. Since all R_X gates in a mixer layer share the same parameter, the parameter vector is $\theta = (\dots, \gamma_k, \beta_k, \dots)$, where γ_k and β_k are the parameters of the k -th cost and mixer layers, respectively. (C) In the VQE ansatz, rotation and entanglement layers alternate. Since the R_Z and R_Y gates in a rotation layer do not share parameters, and entanglement layers use no parameters, the parameter vector is $\theta = (\dots, \theta_{k,1}, \theta_{k,2}, \dots, \theta_{k,7}, \theta_{k,8}, \dots)$, where $\theta_{k,1}, \theta_{k,2}, \dots, \theta_{k,7}, \theta_{k,8}$ are the parameters of the k -th rotation layer.

2.1.2 Variational Quantum Algorithms

One way to solve QUBO is through variational quantum algorithms (VQAs). In essence, these are hybrid algorithms consisting of two parts, a quantum circuit and a classical optimizer, as shown in Fig. 2. The quantum circuit $U(\theta)$ is composed of multiple layers parameterized through rotation gates. For this reason, it is also known as an ansatz. The ansatz is applied to an initial state $|\psi\rangle = |0 \dots 0\rangle$, producing a parameterized state $|\psi(\theta)\rangle = U(\theta)|\psi\rangle$. Measuring $|\psi(\theta)\rangle$ yields an estimate of the expectation value $\langle H_f \rangle$ of a Hamiltonian H_f that encodes the QUBO objective function f . The classical optimizer then minimizes $\langle H_f \rangle$ by tuning and updating the circuit parameters θ . This process repeats until $\langle H_f \rangle$ converges to the minimum value of the problem. Two representative examples are the Quantum Approximate Optimization Algorithm (QAOA) [12] and the Variational Quantum Eigensolver (VQE) [22].

The main difference between QAOA [12] and VQE [22] lies in their ansatzes. In QAOA, the ansatz consists of alternating cost and mixer layers, as shown in Fig. 2. The cost layers are built from the Hamiltonian H_f corresponding to the objective function f of the problem. The mixer layers are built from R_X rotation gates, which rotate each qubit about the X -axis of the Bloch sphere, driving it between $|0\rangle$ and $|1\rangle$, and thereby allowing the search to move between different candidate solutions.

Meanwhile, VQE was originally developed for quantum chemistry [22], where it estimates the ground-state energy of a molecular Hamiltonian. This approach can also be applied to QUBO in the same way. Its ansatz consists of entanglement layers alternating with rotation layers, as shown in Fig. 2. The entanglement layers are composed of controlled gates arranged in a fixed structure. The rotation layers consist of rotation gates that allow exploration of the solution space, as in QAOA. However, in VQE, each gate in the rotation layers has its own parameter, whereas in QAOA the gates within the same layer share a single parameter.

2.1.3 Quantum Annealing

The idea of quantum annealing originated from the work of Kadowaki and Nishimori [23]. The method solves an Ising model that combines both longitudinal and transverse components, where the longitudinal component encodes the QUBO objective function. The amplitude of the transverse component is then gradually decreased to zero. By the adiabatic theorem, if this is done sufficiently slowly, the system remains in its ground state throughout the process and ends in the ground state of the longitudinal component, which corresponds to the optimal solution of the problem. The name annealing reflects this gradual decreasing process.

In practice, quantum annealing is realized on dedicated hardware, most prominently D-Wave’s quantum annealers. The latest generation, the Advantage2 system, provides over 4,400 qubits with increased connectivity, enabling optimization problems with large numbers of variables and constraints [24]. Such annealers have been applied to a wide range of optimization problems, from finance [25] to supply chain [26], and especially to logistics [15, 27], including drone-related problems.

2.2 Drone Scheduling

To coordinate drones for transporting goods, the system must not only compute routes suited to the drone configuration but also schedule and assign those routes to drones so that all deliveries are completed as quickly as possible. In this work, we extend our previous study, QUACOD [21], on solving the drone scheduling problem, illustrated in Fig. 1(B).

2.2.1 Definition

The core of the problem lies in the scheduling rather than the routing. We therefore assume that the list of all routes is precomputed and that each route's completion time is determined. In addition, the number of drones is given, and all drones are assumed to share the same configuration, including the charging time required between consecutive routes.

The drone scheduling problem is then formalized as follows. Let n and q be the number of routes and drones, respectively. The completion time of route i , for $i \in [n]$, is denoted r_i , and the charging time is c . For $i \in [n]$ and $j \in [q]$, let $x_{i,j}$ be a binary variable indicating the assignment of route i to drone j : if $x_{i,j} = 1$, route i is assigned to drone j ; otherwise $x_{i,j} = 0$. Following QUACOD [21], the total time for drone j to complete its assigned routes is given by

$$\sum_{i \in [n]} r_i x_{i,j} + c \left(\sum_{i \in [n]} x_{i,j} - 1 \right). \quad (2)$$

Combining this with the constraint that each route is assigned to one and only one drone, the drone scheduling problem can be stated as

$$\begin{aligned} & \underset{\{x_{i,j}\}_{i \in [n], j \in [q]}}{\text{minimize}} && C_{max} \\ & \text{subject to} && x_{i,j} \in \{0, 1\}, \quad \forall i \in [n], j \in [q], \\ & && \sum_{j \in [q]} x_{i,j} = 1, \quad \forall i \in [n], \\ & && \sum_{i \in [n]} r_i x_{i,j} + c \left(\sum_{i \in [n]} x_{i,j} - 1 \right) \leq C_{max}, \quad \forall j \in [q], \end{aligned} \quad (3)$$

where C_{max} denotes the maximum completion time over all drones.

2.2.2 QUBO Reformulation

Due to the constraints, the optimization problem stated in Eqn. 3 obviously cannot be considered a QUBO, so quantum algorithms cannot be applied directly. An intermediate step, called QUBO reformulation, is therefore required to transform the original problem into QUBO form, as described in QUACOD [21].

First, it applies a heuristic to eliminate the use of C_{max} . Specifically, it computes the sum of the completion times of all drones as T , which is constant regardless of the assignment. Therefore, instead of minimizing C_{max} , it minimizes the sum of squared differences between each drone's completion time and the average value $\frac{T}{q}$.

Second, it applies a penalty technique, following the approach in [11], to embed the constraints into the objective function. The penalty coefficient p is estimated from the problem inputs so that it is large enough to enforce constraint satisfaction, yet small enough that the objective is not dominated by the penalty.

Finally, the QUBO objective function for the drone scheduling problem is expressed as

$$f(\{x_{i,j}\}) = \sum_{j \in [q]} \left(\sum_{i \in [n]} r_i x_{i,j} + c \left(\sum_{i \in [n]} x_{i,j} - 1 \right) - \frac{T}{q} \right)^2 + p \sum_{i \in [n]} \left(\sum_{j \in [q]} x_{i,j} - 1 \right)^2. \quad (4)$$

2.2.3 Quantum Optimization via Coordinate Descent

Algorithm 1 The QUACOD Algorithm [21]

Input: f in Eqn. (4) of nq variables $\mathbf{x} = \{x_{i,j}\}_{i \in [n], j \in [q]}$

Output: $\mathbf{x}^{(k)} = \{x_{i,j}^{(k)}\}_{i \in [n], j \in [q]} = \arg \min_{\mathbf{x}} f(\mathbf{x})$

- 1: Initialize $\{x_{i,j}^{(0)}\}_{i \in [n], j \in [q]}$ and $k \leftarrow 0$
 - 2: **repeat**
 - 3: $k \leftarrow k + 1$
 - 4: Randomly select $N \subset [n]$ and $Q \subset [q]$ satisfying QUACOD's constraints
 - 5: $\{x_{i,j}^{(k)}\}_{i \in [n] \setminus N, j \in [q] \setminus Q} \leftarrow \{x_{i,j}^{(k-1)}\}_{i \in [n] \setminus N, j \in [q] \setminus Q}$
 - 6: $\{x_{i,j}^{(k)}\}_{i \in N, j \in Q} \leftarrow \text{QuanOpt} \left(\{x_{i,j}\}_{i \in N, j \in Q}; f, \{x_{i,j}^{(k)}\}_{i \in [n] \setminus N, j \in [q] \setminus Q} \right)$
 - 7: **until** convergence
 - 8: **return** $\mathbf{x}^{(k)}$
-

As indicated in our previous work [21], solving the QUBO in Eqn. 4 requires a large number of qubits as the number of routes and drones increases. Current quantum hardware, however, typically offers only a few hundred to a few thousand qubits. For instance, IBM's Condor processor reaches 1121 qubits [28], while quantum annealers such as D-Wave's Advantage2 system provide over 4400 qubits [24], among the largest available. This limited qubit count poses a significant challenge. To address it, we proposed QUACOD (Quantum Optimization via Coordinate Descent) [21], an algorithm that enables current hardware to tackle practical problems without waiting for the field to mature.

QUACOD [21] is inspired by coordinate descent [29]. In coordinate descent, at each step, only one variable is optimized while the others remain fixed, and the process repeats until convergence. Building on this idea, QUACOD, as shown in Alg. 1, also

selects variables corresponding to random subsets of routes N and drones Q , ensuring that the number of selected variables does not exceed the number of available qubits. A quantum algorithm QuanOpt is then used to optimize the selected variables, after which QUACOD updates the solution and repeats the process until no further improvement is achieved.

However, several open questions remain for QUACOD [21], one of which is the penalty sensitivity issue. The penalty term cannot be too small, since this would almost eliminate the effect of the constraints and lead to infeasible solutions. Conversely, if the penalty term is too large, it may overwhelm the main objective term, producing feasible but suboptimal solutions. Many works aim to determine the best penalty value. Verma and Lewis [30] provide a theoretical lower bound on the penalty coefficient for general QUBO. Other works analyze penalty parameters for specific problems, such as the maximum k -colorable subgraph problem [31], the multi-knapsack problem [32], and the max- k -cut problem [33]. Another line of work proposes approaches for selecting the penalty automatically [34, 35]. Together, these studies show that analyzing penalty sensitivity is essential.

2.3 Agentic Methods

Since their introduction [36], LLMs, in essence text generation models, have experienced remarkable advancements, progressing from generating simple text to automatically handling increasingly complex tasks such as programming [18], and even acting as independent agents that cooperate with each other and with humans [19, 20]. Such agents have attracted growing research interest [37, 38], aiming to reduce repetitive manual effort and help humans focus on more challenging problems.

For automated design, agentic methods have demonstrated their effectiveness across many works. For example, EoH [39] uses an agent to design heuristics for combinatorial optimization, while ADAS [40] uses an agent to design other agents. Meta-Harness [41] further uses an agent to optimize the harness surrounding a language model. In general, these methods treat a design task as a search problem, in which agents explore candidate solutions through repeated iterative steps of proposal, evaluation, and feedback, ultimately selecting the best one.

In QUACOD [21], selecting an appropriate penalty is difficult due to the large search space and instance-dependence, leading to the use of a hand-crafted function, which may not be efficient. This challenge is analogous to the manual design problems addressed by the agentic methods above. That means, if designing the penalty function is viewed as a search problem, applying LLM-based agents to this task is a promising approach.

3 The Proposed Agentic Quantum Optimization via Coordinate Descent (AQUACOD) Approach

In this section, we propose Agentic Quantum Optimization via Coordinate Descent (AQUACOD), a framework that automatically designs the function determining the penalty coefficient p in Eqn. 4 for the drone scheduling problem, as depicted in Fig. 3. We then present the overall pipeline, including how the best penalty functions are

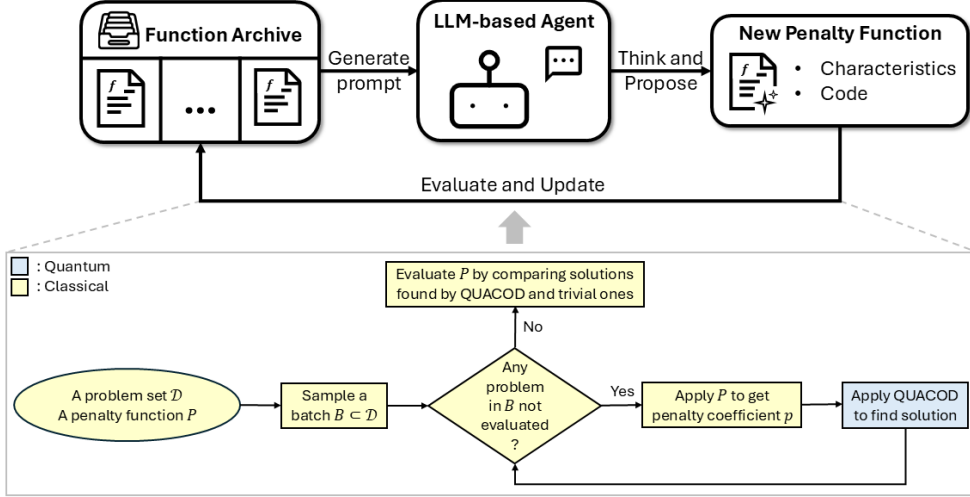


Fig. 3 The Agentic Quantum Optimization via Coordinate Descent framework (top), with the detailed evaluation process of penalty functions (bottom).

selected from the candidate list produced by AQUACOD, as well as the comparison with the baseline.

3.1 Agentic Quantum Optimization via Coordinate Descent

In essence, AQUACOD is a self-refinement loop, as shown in Fig. 3. Its objective is to generate the best penalty functions, which are defined as follows. A penalty function outputs the penalty coefficient p in Eqn. 4, which is later used in QUACOD [21] implement the best penalty functions through Alg. 1. Given the problem parameters, including the number of routes n , the number of drones q , the completion times of all routes $\{r_i\}_{i \in [n]}$, and the charging time c , the best penalty functions should output p^* such that the corresponding solution $\mathbf{x}^*$ is optimal. As shown in Eqn. 4, the value of p depends on n , q , $\{r_i\}_{i \in [n]}$, and c , and thus the penalty function receives them as inputs.

Although the inputs and output are known, one still needs to determine how the best penalty functions are implemented. QUACOD [21] uses a hand-crafted version, defined as follows:

$$p = \sum_{i \in [n]} r_i + cn. \quad (5)$$

This can be expressed as a function in a programming language. Since LLMs have demonstrated strong programming ability [18], AQUACOD uses an LLM as an agent to discover and write penalty functions, rather than relying on a manual approach. Specifically, the agent receives a description of the drone scheduling problem and how QUACOD solves it in Alg. 1 as a prompt, then outputs the penalty function as code.

Given a static prompt, it is unlikely for an LLM to output the best penalty functions at the beginning. As shown in works [39–41], despite their different strategies,

they all store and use prior candidates as additional information to support the design of new ones in subsequent iterations. This approach not only makes the prompt more dynamic, but also helps the agent identify which candidates succeed and which fail, allowing it to tune its output to keep good characteristics and avoid bad ones. Following this idea, AQUACOD uses a similar mechanism, named the function archive, to store all penalty functions previously proposed by the agent or initialized at the start.

To help the agent know which penalty functions work and which do not, an evaluation needs to be built. The evaluation is based on solution quality: the lower the maximum completion time across all drones, the better the solution. However, there are several challenges. First, all solutions should be compared with the optimal one, but the optimal solution is typically unknown, so we use a trivial solution as an alternative. Second, the value of the optimal solution can vary depending on the specific instance of problem parameters, so to standardize it, we use the relative difference (RD), allowing all instances to contribute the same weight. To sum up, assuming that there are N instances and that the solution of assigning route i to drone $((i - 1) \bmod q) + 1$ serves as the trivial one, the RD for the penalty function P is defined as follows:

$$\text{RD}(P) = \frac{1}{N} \sum_{k=1}^N \frac{t_{\text{param}_k}^{P(\text{param}_k)} - t_{\text{param}_k}^{\text{trivial}}}{t_{\text{param}_k}^{\text{trivial}}}, \quad (6)$$

where param_k denotes the problem parameters n , q , $\{r_i\}_{i \in [n]}$, and c for instance k , $t_{\text{param}_k}^{\text{trivial}}$ is the maximum completion time of the trivial solution, and $t_{\text{param}_k}^{P(\text{param}_k)}$ is the maximum completion time of the solution found by QUACOD [21] using the penalty coefficient $p = P(\text{param}_k)$.

Algorithm 2 The AQUACOD Algorithm

Input: A set of problem instances $\mathcal{D}$, a description *desc* of the problem and method

Output: A function archive $\mathcal{P}$

```

1: Initialize a function archive  $\mathcal{P}$  with predefined penalty functions
2: for each  $P \in \mathcal{P}$  do
3:    $eval \leftarrow \text{Evaluate}(P, \mathcal{D})$ 
4:   Store  $(P, eval)$  in  $\mathcal{P}$ 
5: end for
6: repeat
7:    $prompt \leftarrow \text{PromptGenerate}(desc, \mathcal{P})$ 
8:    $P \leftarrow LLM(prompt)$   $\triangleright P$  is the new penalty function
9:    $eval \leftarrow \text{Evaluate}(P, \mathcal{D})$ 
10:  Store  $(P, eval)$  in  $\mathcal{P}$ 
11: until no significant improvements
12: return  $\mathcal{P}$ 

```

The whole loop of AQUACOD is visualized in Fig. 3 and described in more detail in Alg. 2. Specifically, a set of problem instances $\mathcal{D}$ and a function archive $\mathcal{P}$ containing predefined penalty functions are provided at the beginning. Each instance in $\mathcal{D}$

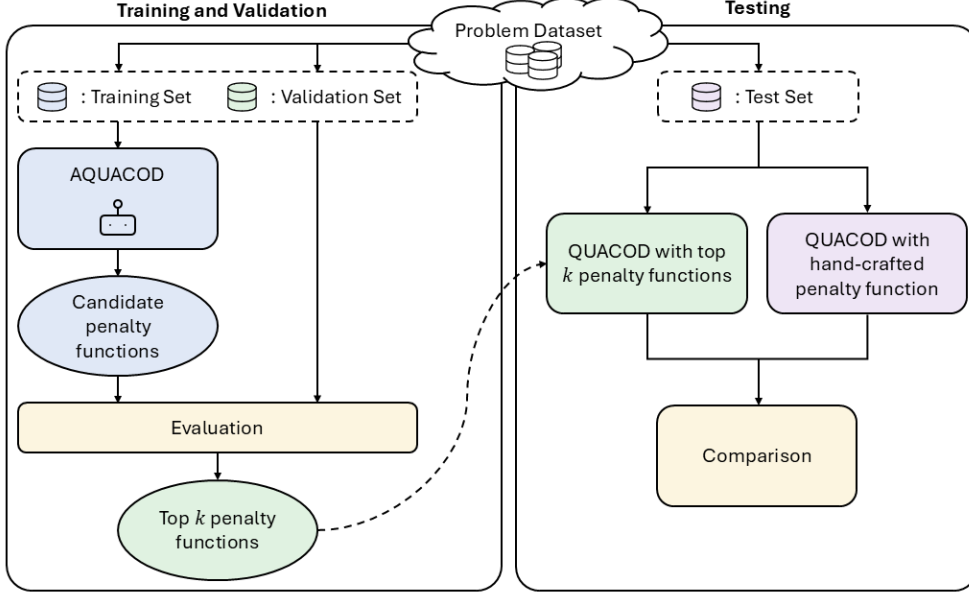


Fig. 4 The overall pipeline for AQUACOD, covering the training and validation phases (left) and the testing phase (right).

contains the specific values of n , q , $\{r_i\}_{i \in [n]}$, and c , while each function in $\mathcal{P}$ contains a list of characteristics describing the function itself, its corresponding implementation code, and its evaluation value on $\mathcal{D}$. The prompt is generated from the problem description, the QUACOD [21] method, and $\mathcal{P}$ as text, then fed into an LLM agent to generate a new penalty function. In addition to keeping good characteristics and excluding bad ones, the agent may add new characteristics to the new penalty function to explore the search space. After evaluation, the new penalty function is added to $\mathcal{P}$, which is later embedded into the prompt to generate the next penalty functions. The process repeats until no significant improvement is observed. Appendix A presents the detailed structure of the prompt. Although RD is the main metric, the evaluation also provides additional diagnostic information (Appendix B), which the agent can use to improve further.

3.2 Overall Pipeline

In this subsection, we describe how AQUACOD is integrated into an overall pipeline, as shown in Fig. 4. A naive application of AQUACOD faces a problem: given a set of problem instances $\mathcal{D}$, the agent may design a penalty function that simply hardcodes the best penalty coefficient for each instance in $\mathcal{D}$. This phenomenon is called overfitting, which is well known in machine learning [42]. To avoid it, the data is split into three datasets, namely the training, validation, and test sets. The training set is used as input to the AQUACOD algorithm to obtain candidate penalty functions in a function archive $\mathcal{P}$. Then, all functions in $\mathcal{P}$ are evaluated on the validation set to choose

the top k penalty functions. On the test set, QUACOD [21] is run once with each of these k functions, and the best of the resulting k solutions is taken. We compare this against the baseline QUACOD with the hand-crafted penalty function in Eqn. 5.

During training, evaluating each function against the entire training set takes significant time. Therefore, we adopt an approach similar to mini-batch gradient descent [42]: we select a small random batch from the training set to evaluate each function. For the validation and test phases, however, we still use the full sets to evaluate functions.

4 Experiments

4.1 Datasets and Configuration

To evaluate AQUACOD, we use the datasets mentioned in QUACOD [21]. Specifically, it contains two datasets, including a standard one and a large-scale one. The standard dataset originally comes from QUADRO [14], which adapts the Augerat dataset [43] for drone problems. The large-scale one is adapted from the Benchmark Problems for Parallel Machine Scheduling dataset [44]. Table 1 lists all problem instances in the two datasets, along with the corresponding number of routes and drones and whether

Table 1 The details of all problem instances in the datasets used in the experiments. Each instance in the standard dataset is used with various numbers of drones, while each one in the large-scale dataset is used with a specific number of drones.

Dataset	Instance	Number of routes	Number of drones	Training	Validation	Testing
Standard	P-N16-K8	7	2-7	✓		
	P-N19-K2	9	2-9		✓	
	P-N20-K2	9	2-9	✓		
	P-N21-K2	11	2-11	✓		
	P-N40-K5	22	2-22			✓
	P-N51-K10	28	2-28	✓		
Large-scale	mj5-200nc1	200	5			✓
	mj10-200nc1		10			✓
	mj20-200nc1		20			✓
	mj50-200nc1		50			✓
	mj5-250nc1	250	5			✓
	mj10-250nc1		10			✓
	mj20-250nc1		20			✓
	mj50-250nc1		50			✓
	mj5-300nc1	300	5			✓
	mj10-300nc1		10			✓
	mj20-300nc1		20			✓
	mj50-300nc1		50			✓
	mj5-500nc1	500	5			✓
	mj10-500nc1		10			✓
	mj20-500nc1		20			✓
	mj50-500nc1		50			✓
	mj5-1000nc1	1000	5			✓
	mj10-1000nc1		10			✓
	mj20-1000nc1		20			✓

Table 2 Comparison on the standard dataset of the maximum time (in hours) that all drones need to complete their assigned routes under the schedules proposed by QUACOD and AQUACOD with the top k penalty functions, for $k = 1$ and $k = 3$. Lower is better.

Drones	Training									Validation			Testing					
	P-N16-K8			P-N20-K2			P-N21-K2			P-N51-K10			P-N19-K2			P-N40-K5		
	QUACOD	AQUACOD		QUACOD	AQUACOD		QUACOD	AQUACOD		QUACOD	AQUACOD		QUACOD	AQUACOD		QUACOD	AQUACOD	
	[21]	$k=1$	$k=3$	[21]	$k=1$	$k=3$	[21]	$k=1$	$k=3$	[21]	$k=1$	$k=3$	[21]	$k=1$	$k=3$	[21]	$k=1$	$k=3$
2	19.90	20.15	19.90	28.35	28.35	28.35	30.60	30.80	30.80	86.45	86.45	86.35	29.55	29.55	29.35	65.05	65.05	65.05
3	13.45	13.45	13.45	18.90	19.10	18.50	21.05	20.50	20.30	57.70	57.80	57.30	20.90	19.90	19.70	43.60	43.25	43.25
4	10.45	10.45	10.45	14.90	13.70	13.70	15.90	15.30	15.10	43.95	43.95	42.85	14.65	14.45	14.45	32.90	33.20	33.05
5	8.45	8.45	8.45	12.05	12.65	12.05	12.05	13.25	12.05	34.40	34.40	34.40	13.25	12.45	12.45	26.45	26.75	26.30
6	8.20	8.20	8.20	10.45	9.85	9.85	10.65	10.65	10.65	28.95	28.95	28.95	11.05	10.65	10.65	22.15	21.75	21.75
7	8.20	8.20	8.20	8.80	8.80	8.80	9.25	9.85	8.65	25.05	24.55	24.55	10.05	9.85	9.85	19.35	18.50	18.50
8	—	—	—	8.80	8.80	8.80	8.60	8.60	8.60	21.75	22.20	21.95	8.60	8.60	8.60	16.45	16.25	16.25
9	—	—	—	8.80	8.80	8.80	8.60	8.60	8.60	19.35	19.10	19.10	8.60	8.60	8.60	15.20	14.75	14.65
10	—	—	—	—	—	—	8.60	8.60	8.60	17.70	17.70	17.70	—	—	—	13.25	13.70	13.05
11	—	—	—	—	—	—	8.60	8.60	8.60	16.10	16.05	15.70	—	—	—	11.80	12.65	12.45
12	—	—	—	—	—	—	—	—	—	14.50	14.95	14.85	—	—	—	11.50	12.45	11.50
13	—	—	—	—	—	—	—	—	—	13.45	13.30	13.25	—	—	—	11.50	11.50	11.50
14	—	—	—	—	—	—	—	—	—	12.45	12.65	12.45	—	—	—	11.50	11.50	11.50
15	—	—	—	—	—	—	—	—	—	11.65	12.05	12.05	—	—	—	11.50	11.50	11.50
16	—	—	—	—	—	—	—	—	—	11.25	11.25	11.25	—	—	—	11.50	11.50	11.50
17-22	—	—	—	—	—	—	—	—	—	10.90	10.90	10.90	—	—	—	11.50	11.50	11.50
23-28	—	—	—	—	—	—	—	—	—	10.90	10.90	10.90	—	—	—	—	—	—

each instance is used for training, validation, or testing. Drone specifications follow QUACOD. In particular, the charging time is 1.25 hours (75 minutes).

We conduct the experiments by simulating 20 qubits on the same workstation, using the same source code and libraries as QUACOD [21]. The quantum optimization setup therefore follows QUACOD exactly. For the agent, we use Claude Sonnet 5 [45] with adaptive extended thinking, a high effort level, and a maximum of 20000 output tokens, keeping all other settings at their default values.

4.2 Evaluation of AQUACOD on Standard and Large-Scale Datasets

We use a part of the standard dataset for training and validation. Initializing with three predefined penalty functions, one of which is the hand-crafted one from QUACOD [21], each iteration of AQUACOD in Alg. 2 generates one penalty function, which is evaluated on a small batch of the training set. The small batch is created by randomly choosing a number of drones for each instance in the training set. After the training phase, we evaluate all functions in the archive on the validation set, using all possible numbers of drones, and retain the top k functions for $k = 1$ and $k = 3$. We then obtain three models: QUACOD with the hand-crafted function; AQUACOD with the top $k = 1$ function; and AQUACOD with the top $k = 3$ functions, where the best of the three resulting solutions is taken, as described in Section 3.2. Table 2 shows results for the whole standard dataset, and Table 3 shows results for the whole large-scale dataset. In both tables, a case is counted as improved when AQUACOD outperforms QUACOD under at least one setting, and as degraded when QUACOD outperforms both settings.

The standard dataset. Since the top k functions are selected based on their performance on the validation set, AQUACOD outperforms QUACOD [21] on 5 of the 8 cases there. Of the remaining three, one improves when k is increased to 3, and the last two remain unchanged because the optimal solution has already been reached. On the unseen instances in the test set, AQUACOD outperforms QUACOD on 5 of

Table 3 Comparison on the large-scale dataset of the maximum time (in minutes) that all drones need to complete their assigned routes under the schedules proposed by QUACOD and AQUACOD with the top k penalty functions, for $k = 1$ and $k = 3$. Lower is better.

Instance	QUACOD [21]	AQUACOD	
		$k = 1$	$k = 3$
mj5-200nc1	5135	5130	5130
mj10-200nc1	2476	2476	2476
mj20-200nc1	1215	1225	1215
mj50-200nc1	471	473	472
mj5-250nc1	6485	6488	6482
mj10-250nc1	3129	3124	3121
mj20-250nc1	1567	1571	1571
mj50-250nc1	612	608	608
mj5-300nc1	7652	7660	7654
mj10-300nc1	3839	3844	3840
mj20-300nc1	1912	1916	1908
mj50-300nc1	733	733	733
mj5-500nc1	12824	12824	12820
mj10-500nc1	6486	6475	6475
mj20-500nc1	3217	3215	3215
mj50-500nc1	1253	1259	1259
mj5-1000nc1	25786	25786	25786
mj10-1000nc1	12817	12824	12816
mj20-1000nc1	6470	6464	6464

21 cases and underperforms on only 2. Increasing k to 3 improves two further cases. Overall, AQUACOD gives better results on 18 of 80 cases with $k = 1$, increasing to 26 with $k = 3$, while underperforming on only 6.

The large-scale dataset. As shown in Table 1, all instances in this dataset serve as a test set, meaning AQUACOD has no prior knowledge of them, which allows us to test its generalizability to large-scale instances. AQUACOD outperforms QUACOD [21] on 6 of 19 cases and underperforms on 5. Increasing k to 3 improves four further cases, giving 10 improvements against 5.

Taken together, the results show that AQUACOD improves on QUACOD [21], with gains on 24 cases and 11 cases degraded, with most gains coming from the standard dataset. Increasing k to 3 yields further improvements, up to 36 cases, at the cost of running QUACOD three times per instance rather than once. The gains are clearer on small-scale instances than on large-scale ones, indicating that the influence of the penalty coefficient decreases as problem size grows.

5 Conclusion

In this work, AQUACOD improves on QUACOD [21], especially on the standard dataset, improving up to 26 cases while degrading only 6. This suggests that agentic methods can automate tasks that are currently performed manually in quantum

algorithms for drone scheduling, and potentially in quantum optimization more broadly.

However, AQUACOD is not a one-size-fits-all solution. The improvements on the large-scale dataset are limited compared to QUACOD [21], suggesting that agentic methods may offer little benefit when the factor being optimized, such as the penalty coefficient, does not strongly influence solution quality. In addition, part of the improvement comes at the cost of increased computation, indicating that AQUACOD is best suited to problems with a low cost per evaluation and a search space that is large and difficult to explore with conventional methods. Identifying similar problems in quantum applications is a promising direction that we plan to pursue in future work.

Appendix A Prompt Structure

To allow the agent to find the best penalty functions, sufficient information must be embedded into the prompt. The prompt is assembled from the string templates shown in Code 1.

Code 1 The string templates used in the prompt for AQUACOD.

```

1 PROBLEM=r"""For a positive integer $k$, let  $[k]$  denote the set
    $\{1, \dots, k\}$ .
2 Let  $n$  and  $q$  be the numbers of routes and drones, respectively
   .
3 The completion time of route  $i$ , for  $i \in [n]$ , is denoted
    $r_i$ , and the recharging time is  $c$ .
4 For  $i \in [n]$  and  $j \in [q]$ , let  $x_{i,j}$  be a binary
   variable indicating the assignment of route  $i$  to drone  $j$ :
   if  $x_{i,j} = 1$ , route  $i$  is assigned to drone  $j$ ;
   otherwise  $x_{i,j} = 0$ .
5 The total time for drone  $j$  to complete its assigned routes is
    $T_j = \sum_{i \in [n]} r_i x_{i,j} + c(\sum_{i \in [n]} x_{i,j} - 1)$ .
6 The maximum completion time over all drones is  $C_{\max} = \max\{T_j \mid j \in [q]\}$ .
7 Note that:
8 - Each route is assigned to one and only one drone:  $\sum_{j \in [q]} x_{i,j} = 1, \forall i \in [n]$ .
9 - Each drone is assigned at least one route, so  $q \leq n$  and
    $T_j \geq 0$  for all  $j \in [q]$ .
10 - The recharging time  $c$  is incurred between two consecutive
   routes of the same drone.
11 The objective is to minimize  $C_{\max}$ ."""
12
13 METHOD=r"""Let  $T$  be the sum of the completion times of all
   drones:  $T = \sum_{j \in [q]} T_j = \sum_{i \in [n]} r_i + c(n - q)$ .
14 The problem is not solved directly, but through a QUBO, and the
   solver built around this QUBO formulation is called QUACOD.

```

```

15 The QUBO objective function for the problem is  $f(\{x_{i,j}\}_{i \in [n], j \in [q]}) = \sum_{j \in [q]} (T_j - \frac{T}{q})^2 + p \sum_{i \in [n]} (\sum_{j \in [q]} x_{i,j} - 1)^2$  where  $p$  is a penalty coefficient.
16 Given  $n$ ,  $q$ ,  $\{r_i\}_{i \in [n]}$ ,  $c$ , and  $p$ , the objective is to minimize  $f(\{x_{i,j}\}_{i \in [n], j \in [q]})$ .
17
18 PENALTY_FUNCTION=r"""The penalty function must be implemented with the following signature:
19 ``python
20 def penalty_function(drone_size: int,
21                     route_costs: list[float],
22                     recharging_time: float) -> float:
23     # WRITE YOUR CODE HERE
24     penalty = ...
25     return penalty
26 ``
27 where the number of routes  $n$  is  $\text{len}(\text{route\_costs})$ , the number of drones  $q$  is  $\text{drone\_size}$ , the completion times of routes  $\{r_i\}_{i \in [n]}$  are  $\text{route\_costs}$ , the recharging time  $c$  is  $\text{recharging\_time}$ , and the penalty coefficient  $p$  is  $\text{penalty}$ .
28 The code must be self-contained and executable, and numpy is the only module allowed to be imported, so the code may start with an import statement such as "import numpy as np".
29 The returned penalty coefficient  $p$  must be a finite positive number.
30
31 FUNCTION_ARCHIVE=r"""The function archive contains a list of penalty functions along with their evaluations, where each function is described as follows:
32 - The field "characteristics" contains a list of strings where each string describes one characteristic of the penalty function.
33 - The field "code" contains a Python code string whose implementation is described in the PENALTY FUNCTION section.
34 - The field "eval" contains, for the penalty function, the average relative difference (RD) between  $C_{\max}$  of the solution found by QUACOD and  $C_{\max}$  of the trivial solution, which assigns route  $i$  to drone  $((i - 1) \bmod q) + 1$  for each  $i \in [n]$ . The lower the "eval", the better the penalty function.
35 - The field "results" contains a list of instances along with their corresponding running results. Each one contains  $n$ ,  $q$ ,  $\{r_i\}_{i \in [n]}$ ,  $c$ ,  $p$ ,  $C_{\max}$  of the QUACOD solution, RD,  $C_{\max}$  of the trivial solution, the ideal result  $\max(\frac{T}{q}, \max_{i \in [n]} r_i)$ , the number of failed trials, the distributions of the values of the objective function and  $C_{\max}$  across all QUACOD iterations. The details are explained in the NOTES section.

```



```

36
37 OUTPUT=r"""The output is the penalty function, and it has the
    following JSON structure:
38 - The field "characteristics" contains a list of strings where
    each string describes one characteristic of the function.
39 - The field "code" contains a Python code string whose
    implementation is described in the PENALTY FUNCTION section.
    """
40
41 NOTES=r"""- Every histogram has 16 uniform bins and is written as
    {bins: [(the left edge of the bin, the number of values
    falling in the bin), ...], max: the right edge of the last bin
    }. The right edge of a bin is the left edge of the next bin.
42 - For an instance with fewer than 100 routes, all route costs  $\{r_i\}_{i \in [n]}$  are provided in full. For an instance with
    100 routes or more, they are provided as a histogram to
    express their distribution.
43 -  $SC_{\max}$  of the QUACOD solution and  $SC_{\max}$  of the trivial
    solution are two distinct quantities. Both of them are used to
    calculate RD.
44 - The ideal result is  $\max(\frac{T}{q}, \max_{i \in [n]} r_i)$ ,
    which is the lowest value that  $SC_{\max}$  can reach: no
    assignment can beat the average load  $\frac{T}{q}$ , nor can it
    beat the longest single route  $\max_{i \in [n]} r_i$ . In
    practice  $SC_{\max}$  may never reach it. The value is provided
    to determine how much room remains to improve  $SC_{\max}$  of the
    QUACOD solution.
45 - The number of failed trials is the number of iterations in
    which the quantum optimizer in QUACOD returns no feasible
    solution. A failed trial indicates that the penalty
    coefficient is set too low. The penalty functions you propose
    should keep the number of failed trials at zero.
46 - The distributions of the values of the objective function and
     $SC_{\max}$  across all QUACOD iterations are provided as
    histograms. This supports analysis of the behavior of the
    penalty coefficient. A bad penalty coefficient keeps the
    values of the objective function and  $SC_{\max}$  high, which
    means the convergence process does not work well."""
47
48 PROMPT=r"""1. DESCRIPTION
49
50 1.1. PROBLEM
51 {PROBLEM}
52
53 1.2. METHOD
54 {METHOD}
55
56 1.3. PENALTY FUNCTION
57
58

```

```

59 {PENALTY_FUNCTION}
60
61
62 1.4. FUNCTION ARCHIVE
63
64 {FUNCTION_ARCHIVE}
65
66 1.5. OUTPUT
67
68 {OUTPUT}
69
70 2. LIST OF PENALTY FUNCTIONS IN FUNCTION ARCHIVE
71
72 {function_archive}
73
74 3. YOUR OBJECTIVE
75
76 You are given the descriptions of the problem, the method, the
    penalty function, the function archive, and the output format,
    together with a list of penalty functions contained in the
    FUNCTION ARCHIVE section. Your objective is to propose a
    penalty function that you think will help QUACOD to minimize
    $C_{{max}}$ for each instance. To do that, you will retain the
    good characteristics of the penalty functions in the FUNCTION
    ARCHIVE section and discard the bad ones, while at the same
    time introducing new characteristics to enable exploration in
    the function space, and then write them in the field "
    characteristics". Based on these characteristics, you will
    write the corresponding penalty function as Python code in the
    field "code".
77
78 4. NOTES
79
80 {NOTES}
81 " " "

```

Appendix B Diagnostic Information for Evaluation

Since RD is calculated by comparing the solution found by QUACOD against the trivial solution rather than the optimal one, it is difficult to tell for which instances the penalty function designed by AQUACOD performs well and brings QUACOD close to the optimum. We therefore provide diagnostic information for each instance as an additional signal for the agent, which it can use when proposing the next penalty functions. This information is shown in Table B1.

Table B1: Diagnostic information provided to the agent for each instance.

Name	Description
n	The number of routes.
q	The number of drones.
$\{r_i\}_{i \in [n]}$	The completion times of all routes.
c	The charging time.
p	The penalty coefficient used in the instance.
QUACOD solution	The maximum completion time of the solution found by QUACOD.
Trivial solution	The maximum completion time of the trivial solution.
RD	The relative difference between the QUACOD and trivial solutions.
Ideal result	$\max(T/q, \max_{i \in [n]} r_i)$, which is the lowest value that a solution can reach. It indicates how much room remains to improve the QUACOD solution.
Failed trials	The number of iterations in which the quantum optimizer in QUACOD returns no feasible solution.
Objective values	The distribution of the values of the objective function across iterations, represented as a histogram.
Completion times	The distribution of the maximum completion times across iterations, represented as a histogram.

References

- [1] Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the Twenty-eighth Annual ACM Symposium on Theory of Computing, pp. 212–219 (1996)
- [2] Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM review **41**(2), 303–332 (1999)
- [3] Nguyen, H.-Q., Pandey, S., Luu, K.: Qlam: A quantum long-attention memory approach to long-sequence token modeling. arXiv preprint arXiv:2605.13833 (2026)
- [4] Nguyen, H.-Q., Nguyen, X.-B., Pandey, S., Khan, S.U., Safro, I., Luu, K.: Qmoe: A quantum mixture of experts framework for scalable quantum neural networks. In: 2025 IEEE International Conference on Quantum Computing and Engineering (QCE), vol. 2, pp. 223–228 (2025). IEEE
- [5] Nguyen, H.-Q., Nguyen, X.B., Chen, S.Y.-C., Churchill, H., Borys, N., Khan, S.U., Luu, K.: Diffusion-inspired quantum noise mitigation in parameterized quantum circuits. Quantum Machine Intelligence **7**(1), 55 (2025)
- [6] Nguyen, X.-B., Nguyen, H.-Q., Churchill, H., Khan, S.U., Luu, K.: Quantum

- visual feature encoding revisited. *Quantum Machine Intelligence* **6**(2), 61 (2024)
- [7] Nguyen, X.-B., Nguyen, H.-Q., Churchill, H., Khan, S.U., Luu, K.: Hierarchical quantum control gates for functional mri understanding. In: 2024 IEEE Workshop on Signal Processing Systems (SiPS), pp. 159–164 (2024). IEEE
 - [8] Nguyen, X.-B., Nguyen, H.-Q., Chen, S.Y.-C., Khan, S.U., Churchill, H., Luu, K.: Qclusformer: A quantum transformer-based framework for unsupervised visual clustering. In: 2024 IEEE International Conference on Quantum Computing and Engineering (QCE), vol. 2, pp. 347–352 (2024). IEEE
 - [9] Holliday, J.B., Morgan, B., Churchill, H., Luu, K.: Hybrid quantum tabu search for solving the vehicle routing problem. In: 2024 IEEE International Conference on Quantum Computing and Engineering (QCE), vol. 2, pp. 353–358 (2024). IEEE
 - [10] Nguyen, X.B., Churchill, H., Luu, K., Khan, S.U.: Quantum vision clustering. arXiv preprint arXiv:2309.09907 (2023)
 - [11] Lucas, A.: Ising formulations of many np problems. *Frontiers in physics* **2**, 74887 (2014)
 - [12] Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm. arXiv preprint arXiv:1411.4028 (2014)
 - [13] Toth, P., Vigo, D.: *Vehicle Routing: Problems, Methods, and Applications*. MOS-SIAM Series on Optimization. Society for Industrial and Applied Mathematics, Philadelphia, PA (2014). <https://doi.org/10.1137/1.9781611973594>
 - [14] Holliday, J.B., Blount, D., Nguyen, H.Q., Khan, S.U., Luu, K.: Quadro: A hybrid quantum optimization framework for drone delivery. In: 2025 IEEE International Conference on Quantum Computing and Engineering (QCE), vol. 1, pp. 2090–2100 (2025). IEEE
 - [15] Holliday, J.B., Osaba, E., Luu, K.: An advanced hybrid quantum tabu search approach to vehicle routing problems. *Quantum Machine Intelligence* **8**(1), 37 (2026)
 - [16] Holliday, J.B., Blount, D., Osaba, E., Luu, K.: Advanced quantum annealing approach to vehicle routing problems with time windows. arXiv preprint arXiv:2503.24285 (2025)
 - [17] Klusch, M., Lässig, J., Müssig, D., Macaluso, A., Wilhelm, F.K.: Quantum artificial intelligence: A brief survey. *KI-Künstliche Intelligenz* **38**(4), 257–276 (2024)
 - [18] Jiang, J., Wang, F., Shen, J., Kim, S., Kim, S.: A survey on large language

- models for code generation. *ACM Transactions on Software Engineering and Methodology* **35**(2), 1–72 (2026)
- [19] Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O’Sullivan, B., Nguyen, H.D.: Multi-agent collaboration mechanisms: A survey of llms. *arXiv preprint arXiv:2501.06322* (2025)
 - [20] Zou, H.P., Huang, W.-C., Wu, Y., Guo, J., Chen, Y., Miao, C., Nguyen, H.H., Zhou, Y., Zhang, W., Fang, L., *et al.*: Llm-based human-agent collaboration and interaction systems: A survey. In: *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 36335–36364 (2026)
 - [21] Nguyen, V.-Q.-H., Nguyen, H.-Q., Khan, S.U., Safro, I., Luu, K.: Quacod: Quantum optimization via coordinate descent for scalable drone scheduling. *arXiv preprint arXiv:2605.14001* (2026)
 - [22] Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.-H., Zhou, X.-Q., Love, P.J., Aspuru-Guzik, A., O’Brien, J.L.: A variational eigenvalue solver on a photonic quantum processor. *Nature communications* **5**(1), 4213 (2014)
 - [23] Kadowaki, T., Nishimori, H.: Quantum annealing in the transverse ising model. *Physical Review E* **58**(5), 5355 (1998)
 - [24] D-Wave Quantum Inc.: Performance gains in the D-Wave Advantage2 system at the 4,400-qubit scale. Technical Report 14-1083A-A, D-Wave Systems Inc. (2025). Accessed: 2026-07-30. <https://www.dwavequantum.com/media/wakjcpf/adv2-4400q-whitepaper-1.pdf>
 - [25] Phillipson, F., Bhatia, H.S.: Portfolio optimisation using the d-wave quantum annealer. In: *International Conference on Computational Science*, pp. 45–59 (2021). Springer
 - [26] Garcia Garcia, J., Galan Jativa, P.: Application of quantum annealing to supply chain planning under uncertainty. In: *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pp. 2216–2223 (2023)
 - [27] Gromiec, J., Kapica, M., Nowicki, M., Uchroński, M., Wasilewski, M.: Solving capacitated vehicle routing problem with equal demand on a d-wave quantum machine. In: *International Conference on Soft Computing Models in Industrial and Environmental Applications*, pp. 357–367 (2026). Springer
 - [28] Choi, C.Q.: Ibm’s quantum leap: The company will take quantum tech past the 1,000-qubit mark in 2023. *IEEE Spectrum* **60**(1), 46–47 (2023)
 - [29] Wright, S.J.: Coordinate descent algorithms. *Mathematical programming* **151**(1), 3–34 (2015)

- [30] Verma, A., Lewis, M.: Penalty and partitioning techniques to improve performance of qubo solvers. *Discrete Optimization* **44**, 100594 (2022)
- [31] Quintero, R., Bernal, D., Terlaky, T., Zuluaga, L.F.: Characterization of qubo reformulations for the maximum k-colorable subgraph problem: R. quintero et al. *Quantum Information Processing* **21**(3), 89 (2022)
- [32] Güney, E., Ehrental, J., Hanne, T.: Qubo formulations and characterization of penalty parameters for the multi-knapsack problem. *IEEE Access* **13**, 47086–47098 (2025)
- [33] Harkness, A., Validi, H., Fakhimi, R., Hicks, I.V., Stein, S., Terlaky, T., Zuluaga, L.F.: Characterizing qubo reformulations of the max-k-cut problem for quantum computing. *arXiv preprint arXiv:2511.01108* (2025)
- [34] Ayodele, M.: Penalty weights in qubo formulations: Permutation problems. In: *European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar)*, pp. 159–174 (2022). Springer
- [35] Liu, J., Moraglio, A.: Evaluation of sensitivity analysis of penalty weights in qubo. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pp. 2596–2599 (2025)
- [36] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
- [37] Dong, Y., Jiang, X., Qian, J., Wang, T., Zhang, K., Jin, Z., Li, G.: A survey on code generation with llm-based agents. *arXiv preprint arXiv:2508.00083* (2025)
- [38] Kumarappan, A., Tiwari, M., Song, P., George, R.J., Xiao, C., *et al.*: Leanagent: Lifelong learning for formal theorem proving. In: *International Conference on Learning Representations*, vol. 2025, pp. 73525–73564 (2025)
- [39] Liu, F., Xialiang, T., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., Zhang, Q.: Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In: *Proceedings of the 41st International Conference on Machine Learning. Proceedings of Machine Learning Research*, vol. 235, pp. 32201–32223 (2024)
- [40] Hu, S., Lu, C., Clune, J.: Automated design of agentic systems. In: *International Conference on Learning Representations*, vol. 2025, pp. 21344–21377 (2025)
- [41] Lee, Y., Nair, R., Zhang, Q., Lee, K., Khattab, O., Finn, C.: Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052* (2026)
- [42] Goodfellow, I., Bengio, Y., Courville, A.: *Deep Learning*. MIT Press, Cambridge,

MA (2016). <http://www.deeplearningbook.org>

- [43] Augerat, P., Belenguer, J.M., Benavent, E., Corberán, A., Naddef, D., Rinaldi, G.: Computational results with a branch and cut code for the capacitated vehicle routing problem. Technical Report RR 949-M, Université Joseph Fourier, Grenoble, France (1995)
- [44] Wang, H., Alidaee, B.: Effective heuristic for large-scale unrelated parallel machines scheduling problems. *Omega* **83**, 261–274 (2019)
- [45] Anthropic: Claude Sonnet 5 System Card. <https://www.anthropic.com/claude-sonnet-5-system-card>. Accessed: 2026-07-30 (2026)